Induced Metric Optimisers

March 17, 2026

Contents

1 Introduction

Thomas [1] introduced a class of optimisers that precondition gradients using the Riemannian metric naturally induced when the loss landscape is embedded in higher-dimensional space. The key idea is to embed the N-dimensional parameter space into $\mathbb{R}^{N+1}$ via the loss function, equip the ambient space with a metric h, and pull h back to obtain a geometry-aware preconditioner. The resulting update automatically damps the effective learning rate in high-curvature regions—acting as a smooth form of gradient clipping—while adding only $O(N)$ overhead.

This document extends that framework in three directions (summarised in Table 1):

(i) **Learnable scalar metric** (Section 4): the inverse base metric γ^{-1} is parameterised by a single learnable scalar s, allowing the overall metric scale to adapt during training.

(ii) **Learnable diagonal metric** (Section 5): each parameter receives its own learnable scale s_i, enabling per-parameter metric adaptation with a principled gauge-fixing (mean-centering) to remove the degeneracy with the global scaling hyperparameter ξ.

(iii) **Off-diagonal ambient metric** (Section 6): the ambient metric h in [1] has no coupling between the parameter dimensions and the loss dimension (the off-diagonal blocks are zero). We relax this by introducing coupling vectors $\boldsymbol{a}$ and $\boldsymbol{b}$, which add a second component to the update direction and enable geometry-induced momentum, weight decay, and other behaviours.

Table 1: Summary of optimiser variants. All share the same geometric construction (embed, pull back, invert via Sherman-Morrison-Woodbury) and are $O(N)$ per step. Sections 4–6 are the new contributions of this document.

Variant	**Base metric** γ^{ij}	**Learnable?**	**Section**
Fixed diagonal	$\gamma\,\delta^{ij}$ (hyperparameter)	No	3
Learnable scalar	$e^{s}\,\delta^{ij}$	Yes ($s \in \mathbb{R}$)	4
Learnable diagonal	$e^{s_i}\,\delta^{ij}$	Yes ($\boldsymbol{s} \in \mathbb{R}^N$)	5
Off-diagonal	$\gamma\,\delta^{ij}$ + off-diag. $\boldsymbol{a}, \boldsymbol{b}$	No	6

Roadmap. Section 2 establishes notation. Section 3 reproduces the core method of [1], including the full algorithms, and defines the practical machinery (momentum, EMA, log-loss variant) used by all subsequent sections. The three extensions follow in Sections 4–6. Section 7 collects implementation details common to all variants, and Section 8 presents benchmark results. Appendix A documents the repository structure and sweep infrastructure.

2 Setup and Notation

Given a model with N parameters $\theta(t) = \{\theta^i(t)\}_{i=1}^N$ and a loss function $\mathcal{L} : \mathbb{R}^N \to \mathbb{R}$, our goal is to find the optimal parameter update $\delta\theta$ at each time step t. That is, in preconditioned gradient flow we have

$$\frac{d\theta^i}{dt} = -g^{ij}\frac{\partial \mathcal{L}}{\partial \theta^j} \implies \delta\theta^i \approx -\eta g^{ij}\frac{\partial \mathcal{L}}{\partial \theta^j} \tag{1}$$

where $\eta \in \mathbb{R}_{>0}$ is the learning rate and g^{ij} is the inverse of the induced metric on the parameter space.

Across all variants in this document the induced metric is obtained by the same geometric construction: we embed parameter space into an ambient space via

$$\phi : \mathbb{R}^N \to \mathbb{R}^{N+1}, \qquad \phi(\theta^1, \ldots, \theta^N) = (\theta^1, \ldots, \theta^N, f(\theta)),$$

where $f : \mathbb{R}^N \to \mathbb{R}$ is a scalar function of the parameters (taken to be the loss $\mathcal{L}$, or a function thereof), and then pull back an ambient bilinear form $h \in \mathbb{R}^{(N+1)\times(N+1)}$ to the parameter space. The variants differ in the choice of h (and, for the log-loss variant, the embedding function f; see below).

We use the shorthand $l_i \coloneqq \partial\mathcal{L}/\partial\theta^i$ for the raw gradient. When a base metric γ_{ij} is present, we write $l^i \coloneqq \gamma^{ij} l_j$ for the metric-raised gradient (i.e. the inverse metric applied to $\boldsymbol{l}$). When $\gamma = \mathbb{1}_N$ (the identity), raising an index has no effect, so $l^i = l_i$ and the two notations coincide. The same index-raising convention applies to any covector: $a^i \coloneqq \gamma^{ij} a_j$, $m^i \coloneqq \gamma^{ij} m_j$, etc.

3 Fixed Diagonal Metric

This section reproduces the core method of [1]: a fixed base metric γ defines the ambient space geometry, and the induced metric on parameter space provides a preconditioner for gradient descent. The geometric derivation, practical algorithms, and all notation established here carry forward to the extensions in Sections 4–6.

3.1 Geometric Derivation

The following derivation is due to Thomas [1]; we reproduce it here to establish notation for the extensions that follow.

The ambient metric in [1] is

$$h = \begin{pmatrix} \gamma & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{pmatrix} \in \mathbb{R}^{(N+1)\times(N+1)}$$

where $\gamma \in \mathbb{R}^{N\times N}$ is symmetric positive-definite (taken to be diagonal or scalar in all our experiments). The induced metric g on the parameter space is given by

$$g_{ij} = h_{\alpha\beta} \frac{\partial\phi^\alpha}{\partial\theta^i} \frac{\partial\phi^\beta}{\partial\theta^j} = \gamma_{ij} + \frac{\partial\mathcal{L}}{\partial\theta^i} \frac{\partial\mathcal{L}}{\partial\theta^j}. \tag{2}$$

Then, to calculate the parameter updates $\delta\theta^i$ we need g^{ij}, the inverse of g_{ij}. We can compute this inverse using the Sherman-Morrison-Woodbury formula:

Sherman-Morrison-Woodbury Formula

Let $A \in \mathbb{R}^{n\times n}$ be an invertible matrix, and let $U, V \in \mathbb{R}^{n\times k}$ be matrices. Then, if $C = A + UV^\top$ is invertible, we have

$$C^{-1} = A^{-1} - A^{-1}U(I_k + V^\top A^{-1} U)^{-1} V^\top A^{-1}.$$

This gives us

$$g^{ij} = \gamma^{ij} - \frac{\gamma^{ik}\gamma^{jl} \frac{\partial\mathcal{L}}{\partial\theta^k} \frac{\partial\mathcal{L}}{\partial\theta^l}}{1 + \gamma^{mn} \frac{\partial\mathcal{L}}{\partial\theta^m} \frac{\partial\mathcal{L}}{\partial\theta^n}} = \gamma^{ij} - \frac{l^i l^j}{1 + l_k l^k} \tag{3}$$

where

$$l_i \coloneqq \frac{\partial\mathcal{L}}{\partial\theta^i} \implies l^i = \gamma^{ij} l_j.$$

Thus, the parameter updates are given by

$$\begin{aligned} \delta\theta^i &= -\eta \left(\gamma^{ij} - \frac{l^i l^j}{1 + l_k l^k} \right) l_j \\ &= -\eta l^i \left(1 - \frac{l_j l^j}{1 + l_k l^k} \right) \\ &= -\frac{\eta\, l^i}{1 + l_k l^k}. \end{aligned} \tag{4}$$

So to summarize, (i) we start with an ambient metric h which induces a metric g on the parameter space, (ii) we compute the inverse g^{ij}, and (iii) we use g^{ij} to compute the parameter updates $\delta\theta^i$.

For a single component with $\gamma = \mathbb{1}_N$, the update $\delta\theta^i/\eta = -l_i/(1 + \sum_k l_k^2)$ depends on both l_i and the other gradients $\{l_k\}_{k\neq i}$. Figure 1 plots this as a function of l_i for several fixed values of $\sum_{k\neq i} l_k^2$.

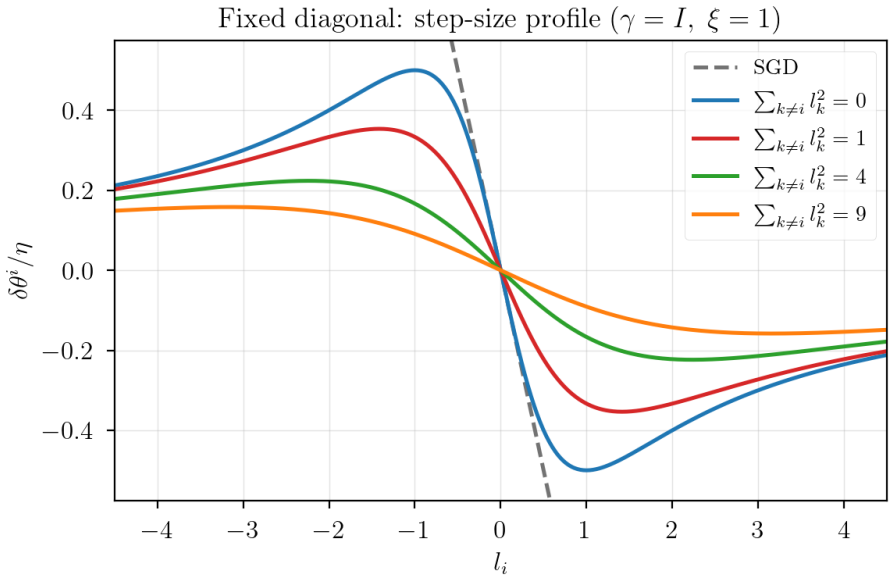

Figure 1: Step-size profile for the fixed diagonal metric ($\gamma = \mathbb{1}$, $\xi = 1$): $\delta\theta^i/\eta = -l_i/(1 + \sum_k l_k^2)$ plotted as a function of l_i for several fixed values of $\sum_{k\neq i} l_k^2$. For small gradients the update is approximately linear (like SGD); for large gradients it saturates, providing smooth gradient clipping. More background curvature from the other parameters produces stronger damping. Compare Figure 2 of [1].

3.2 Practical Algorithm

The theoretical update (4) is clean but not directly usable in stochastic training. Thomas [1] bridges this gap with four modifications, each of which is standard in the optimisation literature but takes on a specific role in the induced-metric framework.

Metric strength ξ. The theoretical derivation implicitly sets $\xi = 1$. Geometrically, ξ enters through the embedding itself: one replaces $\phi(\theta) = (\theta, f(\theta))$ with $\phi(\theta) = (\theta, \sqrt{\xi}\, f(\theta))$, which scales the loss dimension of the ambient space and yields the induced metric $g_{ij} = \gamma_{ij} + \xi\, l_i l_j$. The parameter $\xi > 0$ therefore controls how strongly the loss-surface curvature influences the preconditioner: $\xi \to 0$ recovers the base metric γ^{-1}, while large ξ makes the geometric clipping dominant.

Momentum. An exponential moving average (EMA) of the raw gradients,

$$\boldsymbol{m}_t = \beta_m\, \boldsymbol{m}_{t-1} + (1 - \beta_m)\, \boldsymbol{l}_t, \qquad \boldsymbol{m}_0 = \mathbf{0}, \tag{5}$$

replaces the instantaneous gradient with a smoothed direction. Here $\beta_m \in [0, 1)$ is the momentum decay: $\beta_m = 0$ recovers the raw gradient, while β_m close to 1 gives a long look-back (roughly $1/(1-\beta_m)$ steps). The bias-corrected, metric-raised momentum $\hat{\boldsymbol{m}}_t = \gamma^{-1}\boldsymbol{m}_t/(1-\beta_m^t)$ is then used as the step direction. Momentum is not required by the geometric framework—the induced metric works without it—but it provides the

standard acceleration benefits (variance reduction, faster traversal of flat regions) and is included to match common practice.

Denominator EMA. The scalar $v_t = \xi\, \boldsymbol{l}_t^\top \gamma^{-1} \boldsymbol{l}_t$ is the metric-weighted squared gradient norm. Thomas [1] calls it the *trace statistic* because it equals $\xi \cdot \mathrm{tr}(\gamma^{-1} \boldsymbol{l}\, \boldsymbol{l}^\top)$, the trace of the rank-1 correction to the induced metric. In stochastic training each minibatch (a random subset of the data) yields a different v_t. To stabilise the damping factor, v_t is smoothed with a *second* EMA, independent of the momentum EMA above (momentum smooths the gradient *direction*; this one smooths the scalar *damping*):

$$\bar{v}_t = \beta\, \bar{v}_{t-1} + (1-\beta)\, v_t, \qquad \bar{v}_0 = 0, \tag{6}$$

bias-corrected to $\hat{v}_t = \bar{v}_t/(1-\beta^t)$. The parameter $\beta \in [0, 1)$ plays the same role as β_m: it sets the look-back window, roughly $1/(1-\beta)$ steps. The damping factor is then $r_t = 1/(1+|\hat{v}_t|)$.

Decoupled weight decay. Following [1], weight decay is applied directly to the parameters ($+\lambda\, \theta_{t-1}$ with $\lambda \leq 0$ for decay) rather than through the loss gradient. This decoupled form is the natural choice from the geometric perspective due to general covariance [1]. Like momentum, weight decay is not required by the induced-metric framework but is included because it is standard practice and has a natural geometric justification.

These modifications are collected in Algorithm 1 (adapted from Algorithm 1 of [1]). The log-loss variant is Algorithm 2 (adapted from Algorithm 2 of [1]).

Algorithm 1 Fixed diagonal metric (plain embedding, $f = \mathcal{L}$)

Require: learning rate η, momentum decay β_m, metric strength ξ, denominator EMA decay β, weight decay λ, inverse base metric γ^{-1}

Initialise $t \leftarrow 0$, $\boldsymbol{m}_0 \leftarrow \boldsymbol{0}$, $\bar{v}_0 \leftarrow 0$
while θ_t not converged **do**
  $t \leftarrow t + 1$
  $\boldsymbol{l}_t \leftarrow \nabla_\theta \mathcal{L}(\theta_{t-1})$
  $v_t \leftarrow \xi \cdot \sum_i l_{t,i}\, (\gamma^{-1} \boldsymbol{l}_t)_i$ *// trace statistic*
  $\bar{v}_t \leftarrow \beta\, \bar{v}_{t-1} + (1-\beta)\, v_t$ *// denominator EMA*
  $\hat{v}_t \leftarrow \bar{v}_t/(1-\beta^t)$ *// bias correction*
  $r_t \leftarrow 1/(1+|\hat{v}_t|)$ *// damping factor*
  $\boldsymbol{m}_t \leftarrow \beta_m\, \boldsymbol{m}_{t-1} + (1-\beta_m)\, \boldsymbol{l}_t$ *// momentum*
  $\hat{\boldsymbol{m}}_t \leftarrow \gamma^{-1} \boldsymbol{m}_t/(1-\beta_m^t)$ *// bias-corrected, metric-raised*
  $\theta_t \leftarrow \theta_{t-1} - \eta\, r_t\, \hat{\boldsymbol{m}}_t + \lambda\, \theta_{t-1}$
end while
return θ_t

Algorithm 2 Fixed diagonal metric (log-loss embedding, $f = \log \mathcal{L}$, requires $\mathcal{L} > 0$)

Require: same as Algorithm 1

1: Initialise $t \leftarrow 0$, $\boldsymbol{m}_0 \leftarrow \mathbf{0}$, $\bar{v}_0 \leftarrow 0$
2: **while** θ_t not converged **do**
3: $\quad t \leftarrow t + 1$
4: $\quad \boldsymbol{l}_t \leftarrow \nabla_\theta \mathcal{L}(\theta_{t-1})$
5: $\quad L_t \leftarrow \mathcal{L}(\theta_{t-1})$
6: $\quad v_t \leftarrow \xi \cdot \sum_i l_{t,i} \, (\gamma^{-1} \boldsymbol{l}_t)_i$
7: $\quad \bar{v}_t \leftarrow \beta \, \bar{v}_{t-1} + (1 - \beta) \, v_t$
8: $\quad \hat{v}_t \leftarrow \bar{v}_t / (1 - \beta^t)$
9: $\quad r_t \leftarrow L_t / (L_t^2 + |\hat{v}_t|)$ *// log-loss damping*
10: $\quad \boldsymbol{m}_t \leftarrow \beta_m \, \boldsymbol{m}_{t-1} + (1 - \beta_m) \, \boldsymbol{l}_t$
11: $\quad \hat{\boldsymbol{m}}_t \leftarrow \gamma^{-1} \boldsymbol{m}_t / (1 - \beta_m^t)$
12: $\quad \theta_t \leftarrow \theta_{t-1} - \eta \, r_t \, \hat{\boldsymbol{m}}_t + \lambda \, \theta_{t-1}$
13: **end while**
14: **return** θ_t

In short, the practical update at each step is

$$\delta\theta^i = -\eta \, r_t \, \hat{m}^i, \qquad r_t = \frac{1}{1 + |\hat{v}_t|}, \qquad v_t = \xi \, \gamma^{mn} \, l_{t,m} \, l_{t,n}, \tag{7}$$

where $\hat{v}_t$ is the bias-corrected EMA of v_t (lines 5–8 of Algorithm 1) and $\hat{m}^i$ is the bias-corrected, metric-raised momentum (line 10).

3.3 Log-Loss Variant

Log-Loss Derivation Sketch

With $f = \log \mathcal{L}$ the embedding gradient is $\partial f / \partial \theta^i = l_i / \mathcal{L}$, so the induced metric becomes $g_{ij} = \gamma_{ij} + \xi \, (l_i / \mathcal{L})(l_j / \mathcal{L})$. Applying Sherman-Morrison-Woodbury gives the inverse metric

$$g^{ij} = \gamma^{ij} - \frac{\xi \, l^i l^j}{\mathcal{L}^2 + \xi \, l_k l^k}.$$

Contracting with the *raw* gradient l_j yields $g^{ij} l_j = l^i \cdot \mathcal{L}^2 / (\mathcal{L}^2 + v_t)$, where $v_t = \xi \, \gamma^{mn} \, l_m \, l_n$.

The full geometric update, however, uses the embedding gradient $\partial_j f = l_j / \mathcal{L}$, not the raw gradient:

$$g^{ij} \frac{\partial f}{\partial \theta^j} = \frac{g^{ij} l_j}{\mathcal{L}} = l^i \cdot \frac{\mathcal{L}}{\mathcal{L}^2 + v_t}.$$

The practical damping factor $r_t = \mathcal{L}_t / (\mathcal{L}_t^2 + |\hat{v}_t|)$ (line 7 of Algorithm 2) therefore follows directly from the geometry of the log embedding. This provides stronger damping than $\mathcal{L}^2 / (\mathcal{L}^2 + |\hat{v}_t|)$ as $\mathcal{L} \to 0$, preventing overshooting near minima.

In practice, the update direction $\boldsymbol{r}$ and the metric direction $\boldsymbol{l}$ need not coincide; see Section 7 for the full treatment.

3.4 Implementation

- JAX/Optax: `optimisers/jax_fixed.py`

- PyTorch: `optimisers/torch_fixed.py`
- Equivalence check: `optimisers/compare_fixed.py`

Exposed optimisers: `custom_sgd` (plain embedding, $f = \mathcal{L}$), `custom_sgd_log` (log-loss embedding, $f = \log \mathcal{L}$; requires $\mathcal{L} > 0$), `custom_sgd_rms` (RMS-normalised gradients, i.e. the update direction is $l_i/\|\boldsymbol{l}\|$ instead of l_i). The sweep name is `sgd_metric` / `sgd_log_metric` / `sgd_rms`.

4 Learnable Scalar Metric

The fixed diagonal variant treats γ as a hyperparameter chosen before training. A natural next step is to let the metric adapt during optimisation. We begin with the simplest case: a single learnable scalar that controls the global scale of γ^{-1}.

4.1 Construction

We parameterise the inverse metric as

$$\gamma^{ij} = \alpha\, \delta^{ij}, \qquad \alpha = e^s,$$

where $s \in \mathbb{R}$ is a single learnable scalar. (We learn the *inverse* metric directly because the update rule requires γ^{-1}.) Initialising $s = 0$ gives $\gamma^{-1} = \mathbb{1}_N$, so the optimiser starts as the unscaled fixed diagonal variant.

The ambient metric remains block-diagonal,

$$h = \begin{pmatrix} e^{-s}\,\mathbb{1}_N & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{pmatrix},$$

and the induced metric on parameter space is

$$g_{ij} = e^{-s}\,\delta_{ij} + l_i l_j.$$

Applying the Sherman-Morrison-Woodbury formula with $A_{ij} = e^{-s}\delta_{ij}$, $u = v = \boldsymbol{l}$:

$$g^{ij} = \gamma^{ij} - \frac{\gamma^{ik} l_k\, \gamma^{jl} l_l}{1 + \gamma^{mn} l_m l_n} = e^s\, \delta^{ij} - \frac{e^{2s}\, l_i\, l_j}{1 + e^s \|\boldsymbol{l}\|^2}. \tag{8}$$

The parameter update is therefore

$$\delta\theta^i = -\eta\, g^{ij} l_j = -\frac{\eta\, \gamma^{ij} l_j}{1 + \gamma^{mn} l_m l_n} = -\frac{\eta\, e^s\, l_i}{1 + e^s \|\boldsymbol{l}\|^2}. \tag{9}$$

The denominator depends on s through $\gamma^{mn} = e^s \delta^{mn}$. Figure 2 shows how varying s reshapes the step-size profile relative to the fixed diagonal case (Figure 1).

The practical update follows the same structure as (7), with $\gamma^{ij} = \alpha\, \delta^{ij}$ and the trace statistic specialised to

$$v_t = \xi\, \alpha\, \|\boldsymbol{l}_t\|^2. \tag{10}$$

4.2 Metric Learning

We want to learn s so that $\alpha = e^s$ is calibrated to the local geometry of the loss. The trace statistic $v_t = \xi\alpha\|\boldsymbol{l}_t\|^2$ appears directly in the denominator of the update rule through $r_t = 1/(1 + |\hat{v}_t|)$. When v_t is near zero the damping is inactive and the optimiser reduces to scaled SGD; when v_t is large the damping dominates and the geometric clipping is fully engaged. We adapt s by performing *gradient ascent* on v_t with respect to s. This is a convenient heuristic—v_t is the quantity that already appears in the denominator, and ascent on it has a clear geometric interpretation (see below)—but we do not claim it is the unique or optimal choice of surrogate. Concretely:

$$\frac{\partial v_t}{\partial s} = \xi\, e^s\, \|\boldsymbol{l}_t\|^2 = v_t.$$

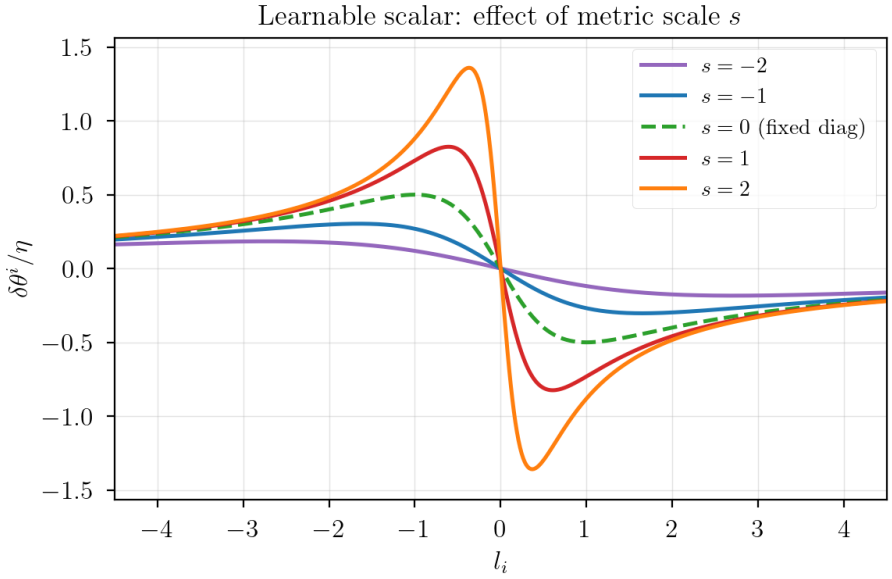

Figure 2: Step-size profile for the learnable scalar metric: $\delta\theta^i/\eta = -e^s l_i/(1+\xi\, e^s\, l_i^2)$ in the single-parameter case ($N = 1$, $\xi = 1$). Increasing s amplifies the peak step size and shifts it toward smaller gradients; decreasing s flattens the curve, approaching plain SGD. The dashed line is the fixed diagonal baseline ($s = 0$).

Why ascent? Consider a single parameter with $\gamma = 1$. The effective step size per unit gradient is

$$\frac{|\delta\theta|}{|l|} = \frac{e^s}{1+\xi\, e^s\, l^2}. \tag{11}$$

Its derivative with respect to the metric parameter s is

$$\frac{\partial}{\partial s}\left(\frac{e^s}{1+\xi\, e^s\, l^2}\right) = \frac{e^s}{(1+\xi\, e^s\, l^2)^2} > 0. \tag{12}$$

The step size is therefore *monotonically increasing* in s (note: increasing in s, not in l), bounded above by $1/(\xi\, l^2)$. Gradient ascent on v increases s, pushing the step size toward this upper bound—the regime where the geometric clipping is most active. Gradient *descent* would decrease s, collapsing the step size toward zero and disabling the metric entirely. The regularisation term (below) provides a restoring force, and the equilibrium between ascent and regularisation determines the operating point.

Regularisation and stability. Since $v_t \geq 0$, the unregularised ascent rule monotonically increases s. Three mechanisms keep s bounded: (i) an L2 regularisation term $-\mu\lambda_s s$ that pulls s back towards zero (i.e. $\gamma^{-1} = \mathbb{1}$), (ii) hard clipping of s to $[-s_{\max}, s_{\max}]$, and (iii) the denominator EMA, which smooths transient gradient spikes.

The online metric update is

$$s \;\leftarrow\; s + \mu\, v_t - \mu\, \lambda_s\, s, \tag{13}$$

where $\mu > 0$ is the metric learning rate and $\lambda_s > 0$ is the L2 regularisation coefficient, followed by clipping to $[-s_{\max}, s_{\max}]$.

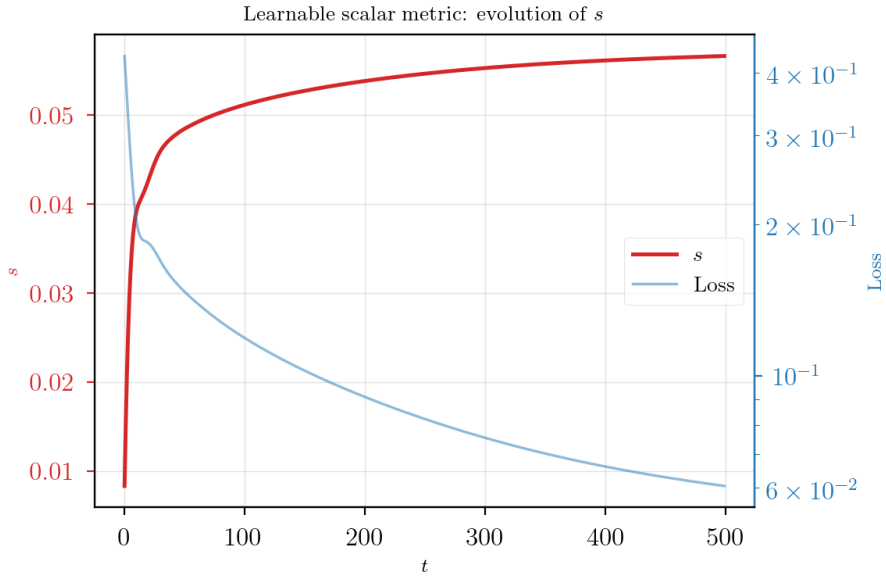

Figure 3: Evolution of the learnable scalar s during training (2-layer MLP on synthetic regression, 500 steps). s rises rapidly in early training as the metric engages with the gradient structure, then plateaus as the regularisation balances the ascent pressure. The loss (right axis, log scale) decreases throughout.

4.3 Algorithm

Algorithm 3 Learnable scalar metric (plain embedding)

Require: all hyperparameters from Algorithm 1, plus metric learning rate μ, metric regularisation λ_s, clipping bound $s_{\max}$

1: Initialise $t \leftarrow 0$, $\boldsymbol{m}_0 \leftarrow \mathbf{0}$, $\bar{v}_0 \leftarrow 0$, $s \leftarrow 0$
2: **while** θ_t not converged **do**
3: $\quad t \leftarrow t + 1$
4: $\quad \alpha \leftarrow e^s$
5: $\quad \boldsymbol{l}_t \leftarrow \nabla_\theta \mathcal{L}(\theta_{t-1})$
6: $\quad v_t \leftarrow \xi\, \alpha\, \|\boldsymbol{l}_t\|^2$
7: $\quad \bar{v}_t \leftarrow \beta\, \bar{v}_{t-1} + (1-\beta)\, v_t$
8: $\quad \hat{v}_t \leftarrow \bar{v}_t / (1-\beta^t)$
9: $\quad r_t \leftarrow 1/(1+|\hat{v}_t|)$
10: $\quad \boldsymbol{m}_t \leftarrow \beta_m\, \boldsymbol{m}_{t-1} + (1-\beta_m)\, \boldsymbol{l}_t$
11: $\quad \hat{\boldsymbol{m}}_t \leftarrow \alpha\, \boldsymbol{m}_t / (1-\beta_m^t)$
12: $\quad \theta_t \leftarrow \theta_{t-1} - \eta\, r_t\, \hat{\boldsymbol{m}}_t + \lambda\, \theta_{t-1}$ *// parameter update*
13: $\quad s \leftarrow s + \mu\, v_t - \mu\, \lambda_s\, s$ *// metric update*
14: $\quad s \leftarrow \mathrm{clip}(s, -s_{\max}, s_{\max})$
15: **end while**
16: **return** θ_t

The log-loss variant replaces line 6 with $r_t \leftarrow L_t / (L_t^2 + |\hat{v}_t|)$.

4.4 Hyperparameters

The learnable scalar variant introduces three new hyperparameters beyond those of the fixed diagonal variant:

- μ (`metric_lr`): step size for the online metric update in (13).
- λ_s (`metric_reg`): L2 regularisation strength on s, controlling how far the metric drifts from identity.
- $s_{\max}$ (`metric_clip`): clipping bound on s, bounding $\alpha \in [e^{-s_{\max}}, e^{s_{\max}}]$.

4.5 Implementation

- JAX/Optax: `optimisers/jax_learnable_scalar.py`
- PyTorch: `optimisers/torch_learnable_scalar.py`
- Equivalence check: `optimisers/compare_learnable_scalar.py`

The plain variant is `custom_sgd_learnable_scalar` (sweep: `sgd_learn_scalar`). The log-loss variant is `custom_sgd_log_learnable_scalar` (sweep: `sgd_learn_scalar_log`).

5 Learnable Diagonal Metric

The learnable scalar metric has a single degree of freedom and therefore applies the same scale to every parameter. We now generalise to a per-parameter learnable scale, giving each parameter its own adaptive metric entry.

5.1 Construction

We promote s to a full vector $\boldsymbol{s} \in \mathbb{R}^N$ with one entry per parameter, giving the per-parameter inverse metric

$$\gamma^{ij} = e^{s_i}\,\delta^{ij}.$$

The ambient metric is

$$h = \begin{pmatrix} \mathrm{diag}(e^{-\boldsymbol{s}}) & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{pmatrix},$$

and the induced metric on parameter space is

$$g_{ij} = e^{-s_i}\,\delta_{ij} + l_i l_j.$$

This is again a rank-1 update to a diagonal matrix. Applying Sherman-Morrison-Woodbury with $A_{ij} = e^{-s_i}\delta_{ij}$ and $u = v = \boldsymbol{l}$,

$$g^{ij} = e^{s_i}\,\delta^{ij} - \frac{l^i\,l^j}{1 + l_k l^k},$$

where $l^i = \gamma^{ij} l_j = e^{s_i} l_i$ (no sum on i), so $l^i l^j = e^{s_i} l_i \cdot e^{s_j} l_j$ and $l_k l^k = \sum_k e^{s_k} l_k^2$. The update is

$$\delta\theta^i = -\eta\, g^{ij} l_j = -\eta \left(l^i - \frac{l^i\,(l_k l^k)}{1 + l_k l^k} \right) = -\frac{\eta\, l^i}{1 + l_k l^k}. \tag{14}$$

With ξ, momentum, and the EMA damping, the practical update is

$$\delta\theta^i = -\eta\, r_t\, \hat{m}^i, \qquad r_t = \frac{1}{1 + |\hat{v}_t|}, \qquad v_t = \xi \sum_k e^{s_k}\, l_k^2, \tag{15}$$

with the same log-loss variant as before. Compared to the scalar case, each parameter θ^i now has an individual scale e^{s_i}, allowing the metric to adapt separately to different regions of parameter space.

5.2 Metric Learning

The per-parameter metric update treats each s_i independently. We use the same surrogate-ascent strategy as in Section 4.2, now applied to the trace statistic $v_t = \xi \sum_k e^{s_k} l_k^2$ which serves as the surrogate objective:

$$J(\boldsymbol{s}) = v_t = \xi \sum_k e^{s_k}\, l_k^2.$$

This is the quantity that appears in the denominator of the update rule, so adapting $\boldsymbol{s}$ to calibrate v_t keeps the damping in a useful operating range (see Section 4.2 for the detailed argument). Its gradient with respect to s_i is

$$\frac{\partial J}{\partial s_i} = \xi\, e^{s_i}\, l_i^2.$$

The online update is therefore

$$s_i \;\leftarrow\; s_i + \mu\, \xi\, e^{s_i}\, l_i^2 - \mu\, \lambda_s\, s_i, \tag{16}$$

followed by mean-centering and clipping (discussed below). The motivation for gradient *ascent* (not descent) and the role of regularisation are the same as in Section 4.2.

5.3 Mean-Centering

Parameter tensors and leaves. A model's parameters are organised as a collection of tensors (e.g. one weight matrix and one bias vector per layer). Each such tensor is called a *leaf*. We write N_ℓ for the number of scalar entries in a single leaf, as distinct from the total parameter count $N = \sum_\ell N_\ell$.

The centering operation. The vector $\boldsymbol{s}$ has one entry per parameter, $i = 1, \ldots, N$. After the gradient step (16), we mean-centre $\boldsymbol{s}$ *within each leaf independently*: for each leaf ℓ containing parameters $i \in I_\ell$ (with $|I_\ell| = N_\ell$),

$$s_i \;\leftarrow\; s_i - \bar{s}_\ell, \qquad \bar{s}_\ell = \frac{1}{N_\ell} \sum_{k \in I_\ell} s_k.$$

This is not an ad-hoc trick but a principled normalisation arising from a gauge degeneracy.

Gauge degeneracy. Write each log-scale as a mean plus a deviation: $s_k = \bar{s} + \delta_k$. By construction $\sum_k \delta_k = \sum_k (s_k - \bar{s}) = N_\ell \bar{s} - N_\ell \bar{s} = 0$. The trace statistic then factorises as

$$v = \xi \sum_k e^{s_k} l_k^2 = \xi\, e^{\bar{s}} \sum_k e^{\delta_k} l_k^2.$$

The global mean $\bar{s}$ acts as a multiplicative prefactor that is exactly degenerate with ξ: shifting all s_k by a constant c is equivalent to scaling ξ by e^c and leaving $\boldsymbol{s}$ unchanged. Mean-centering enforces $\bar{s} = 0$ (note: subsequent clipping may re-introduce a small nonzero mean), which fixes this gauge and ensures that ξ remains the single scalar controlling the overall metric magnitude while $\boldsymbol{s}$ captures only the *relative* per-parameter geometry. Since we apply this per leaf rather than globally, a separate mean mode is removed in each parameter tensor.

Fixed Diagonal as a Special Case

In a highly symmetric special case: if all s_i are initialised to the same value and the gradients have the same magnitude for all parameters at every step, then $\boldsymbol{s}$ stays uniform and mean-centering keeps it at zero. In that case $e^{s_i} = 1$ for all i and $v = \xi \|\boldsymbol{l}\|^2$, which reduces exactly to the fixed diagonal variant with $\gamma = \mathbb{1}_N$. The learnable diagonal metric therefore generalises fixed diagonal by learning *relative* per-parameter scalings, while mean-centering intentionally removes the global scale mode

(which is degenerate with ξ). To recover a learnable global scale, disable mean-centering or make ξ itself learnable.

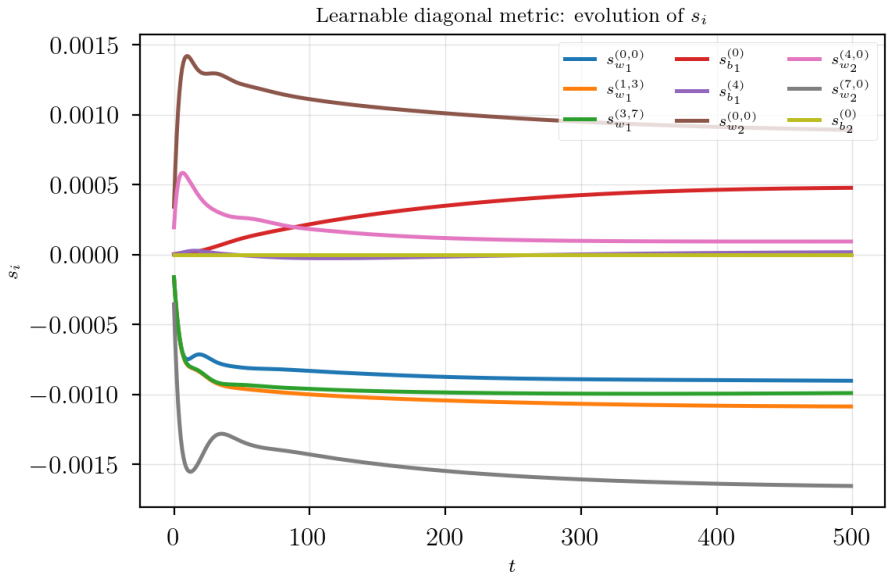

Figure 4: Evolution of per-parameter log-scales s_i during training (same setup as Figure 3). Different parameters acquire different scales: some are upscaled ($s_i > 0$), others downscaled ($s_i < 0$). Mean-centering keeps the average at zero, so $\boldsymbol{s}$ captures only relative geometry.

5.4 Algorithm

Algorithm 4 Learnable diagonal metric (plain embedding)

Require: all hyperparameters from Algorithm 1, plus μ, λ_s, $s_{\max}$

Initialise $t \leftarrow 0$, $\boldsymbol{m}_0 \leftarrow \mathbf{0}$, $\bar{v}_0 \leftarrow 0$, $\boldsymbol{s} \leftarrow \mathbf{0}$
while θ_t not converged **do**
  $t \leftarrow t + 1$
  $\boldsymbol{l}_t \leftarrow \nabla_\theta \mathcal{L}(\theta_{t-1})$
  $v_t \leftarrow \xi \sum_k e^{s_k} \, l_{t,k}^2$
  $\bar{v}_t \leftarrow \beta \, \bar{v}_{t-1} + (1 - \beta) \, v_t$
  $\hat{v}_t \leftarrow \bar{v}_t / (1 - \beta^t)$
  $r_t \leftarrow 1 / (1 + |\hat{v}_t|)$
  $\boldsymbol{m}_t \leftarrow \beta_m \, \boldsymbol{m}_{t-1} + (1 - \beta_m) \, \boldsymbol{l}_t$
  $\hat{m}_t^i \leftarrow e^{s_i} \, m_{t,i} / (1 - \beta_m^t)$
  $\theta_t \leftarrow \theta_{t-1} - \eta \, r_t \, \hat{\boldsymbol{m}}_t + \lambda \, \theta_{t-1}$ *// parameter update*
  $s_i \leftarrow s_i + \mu \, \xi \, e^{s_i} \, l_{t,i}^2 - \mu \, \lambda_s \, s_i$ for each i *// metric gradient ascent*
  $s_i \leftarrow s_i - \bar{s}$ per leaf *// mean-centre (gauge fix)*
  $\boldsymbol{s} \leftarrow \mathrm{clip}(\boldsymbol{s}, -s_{\max}, s_{\max})$
end while
return θ_t

5.5 Implementation

- JAX/Optax: `optimisers/jax_learnable_diag.py`
- PyTorch: `optimisers/torch_learnable_diag.py`
- Equivalence check: `optimisers/compare_learnable_diag.py`

The plain variant is `custom_sgd_learnable_diag` (sweep: `sgd_learn_diag`). The log-loss variant is `custom_sgd_log_learnable_diag` (sweep: `sgd_learn_diag_log`).

6 Off-Diagonal Metric

All preceding variants use an ambient metric h whose off-diagonal blocks (between the N parameter dimensions and the single loss dimension) are zero. We now explore what happens when those blocks are nonzero—a direction explicitly suggested by [1].

6.1 Derivation

Consider a more general ambient bilinear form

$$h = \begin{pmatrix} \gamma & \boldsymbol{a} \\ \boldsymbol{b}^\top & 1 \end{pmatrix} \in \mathbb{R}^{(N+1)\times(N+1)}$$

where $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^N$. Intuitively, the vectors $\boldsymbol{a}$ and $\boldsymbol{b}$ introduce off-diagonal couplings between the parameter space and the loss function dimension. In general, if $\boldsymbol{a} \neq \boldsymbol{b}$ this h is not symmetric, so here we simply treat it as a preconditioning matrix rather than a Riemannian metric.

Stability note. If the effective preconditioner is non-symmetric ($\boldsymbol{a} \neq \boldsymbol{b}$), the update direction need not be a descent direction even for convex objectives. To see why, note that a preconditioned gradient step $\delta\theta = -\eta\, P\, \nabla\mathcal{L}$ is a descent direction only if $\nabla\mathcal{L}^\top P\, \nabla\mathcal{L} > 0$, which is guaranteed when P is positive-definite (or has positive-definite symmetric part). A non-symmetric h can produce a non-symmetric $P = g^{-1}$, for which the quadratic form $\nabla\mathcal{L}^\top P\, \nabla\mathcal{L}$ can be negative—meaning the update moves *uphill*. We therefore recommend defaulting to the symmetric case $\boldsymbol{a} = \boldsymbol{b}$; if using $\boldsymbol{a} \neq \boldsymbol{b}$, consider symmetrising the effective preconditioner or otherwise enforcing positive-definiteness.

The induced bilinear form on the parameter space is

$$\begin{aligned} g_{ij} &= h_{\alpha\beta} \frac{\partial \phi^\alpha}{\partial \theta^i} \frac{\partial \phi^\beta}{\partial \theta^j} \\ &= \gamma_{ij} + \xi(a_i l_j + b_j l_i + l_i l_j)\,. \end{aligned}$$

To compute the inverse g^{ij}, we notice that g can be expressed as a rank-2 update to γ: for $U = [\boldsymbol{a}, \boldsymbol{l}]$, $V = [\boldsymbol{l}, \boldsymbol{b} + \boldsymbol{l}] \in \mathbb{R}^{N\times 2}$ we have

$$\begin{aligned} \gamma_{ij} + \xi\, U_{ik} V_{jk} &= \gamma_{ij} + \xi(a_i l_j + (b_j + l_j) l_i) \\ &= \gamma_{ij} + \xi(a_i l_j + b_j l_i + l_i l_j) \\ &= g_{ij}. \end{aligned}$$

Applying the Sherman-Morrison-Woodbury formula to $g = \gamma + \xi\, UV^\top$:

$$g^{ij} = \gamma^{ij} - \xi\, \gamma^{ik} U_{k\alpha} \Big(\delta_{\alpha\beta} + \xi\, V_{m\alpha} \gamma^{mn} U_{n\beta}\Big)^{-1} V_{l\beta} \gamma^{lj}.$$

For convenience we define the scalar contractions

$$c_{al} \coloneqq a_n l^n, \quad c_{ll} \coloneqq l_n l^n, \quad c_{ba} \coloneqq b_n a^n, \quad c_{bl} \coloneqq b_n l^n,$$

so that the 2×2 core matrix is

$$\mathbb{1}_2 + \xi\, V^\top \gamma^{-1} U = \begin{pmatrix} 1 + \xi\, c_{al} & \xi\, c_{ll} \\ \xi\,(c_{ba} + c_{al}) & 1 + \xi\,(c_{bl} + c_{ll}) \end{pmatrix}$$

with determinant

$$D = 1 + \xi(c_{al} + c_{bl} + c_{ll}) + \xi^2 (c_{al}\, c_{bl} - c_{ll}\, c_{ba})\,. \tag{17}$$

Inverting the 2×2 system and contracting yields the parameter update

$$\delta\theta^i = -\eta\, g^{ij} l_j = -\eta \left[\frac{1 + \xi\, c_{al}}{D}\, l^i - \frac{\xi\, c_{ll}}{D}\, a^i \right]. \tag{18}$$

The closed-form inverse requires $D \neq 0$ (equivalently, $\det(\mathbb{1}_2 + \xi\, V^\top \gamma^{-1} U) \neq 0$). The JAX implementation includes a near-singular fallback; the PyTorch implementation solves the 2×2 system directly via `torch.linalg.solve`.

A numerical check of this has been performed in documents/off-diag-check.ipynb: for random $\boldsymbol{a}, \boldsymbol{b}, \gamma, \boldsymbol{l}$ the closed-form $g^{ij} l_j$ matches `torch.linalg.solve`$(g, \boldsymbol{l})$ within floating-point tolerance.

Sanity Checks

Diagonal recovery ($\boldsymbol{a} = \boldsymbol{b} = \boldsymbol{0}$): $c_{al} = c_{ba} = c_{bl} = 0$, so $D = 1 + \xi\, c_{ll}$. Equation (18) gives $\delta\theta^i = -\eta\, l^i/(1 + \xi\, c_{ll})$, which matches the fixed diagonal case. ✓

Symmetric coupling ($\boldsymbol{a} = \boldsymbol{b} = \boldsymbol{l}$): $c_{al} = c_{bl} = c_{ba} = c_{ll}$, so $D = 1 + 3\xi\, c_{ll} + \xi^2(c_{ll}^2 - c_{ll}^2) = 1 + 3\xi\, c_{ll}$. The update simplifies to $\delta\theta^i = -\eta\, l^i/(1 + 3\xi\, c_{ll})$. ✓

One-sided coupling ($\boldsymbol{a} = \boldsymbol{0}$, $\boldsymbol{b} \neq \boldsymbol{0}$): $c_{al} = c_{ba} = 0$, $D = 1 + \xi(c_{bl} + c_{ll})$. The update reduces to $\delta\theta^i = -\eta\, l^i/D$, i.e. a scaled gradient step whose denominator depends on $\boldsymbol{b}$ through c_{bl}.

The update direction (18) has two components: one along l^i (the gradient direction) and one along a^i (the off-diagonal coupling direction). Depending on the choices of $\boldsymbol{a}$ and $\boldsymbol{b}$, this can lead to a variety of behaviours:

- If $\boldsymbol{a} = \boldsymbol{0}$ or $\boldsymbol{a} \propto \boldsymbol{l}$: plain gradient descent with an adaptive learning rate.
- If $\boldsymbol{a} =$ momentum: a momentum-like update with an adaptive learning rate.
- If $\boldsymbol{a} = \boldsymbol{\theta}$ (the parameter vector): gradient descent combined with a form of geometry-induced weight decay or growth, depending on the sign of $\xi\, c_{ll}/D$.

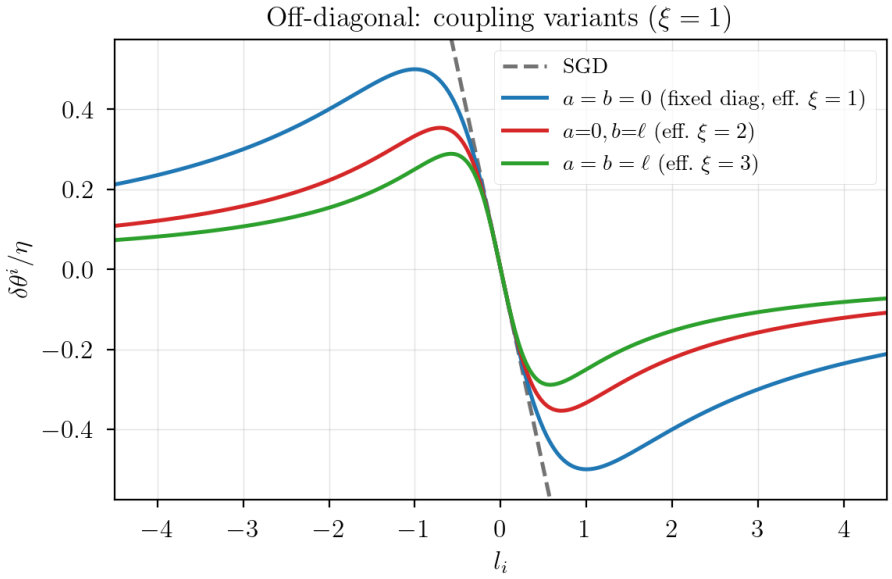

Figure 5: Step-size profiles for off-diagonal variants in the single-parameter case ($N = 1$, $\gamma = 1$, $\xi = 1$). All curves plot $\delta\theta/\eta = -l/(1 + \xi_{\text{eff}}\, l^2)$ where ξ_{eff} depends on the coupling: $\xi_{\text{eff}} = 1$ (fixed diagonal, $\boldsymbol{a} = \boldsymbol{b} = 0$), $\xi_{\text{eff}} = 2$ (one-sided), $\xi_{\text{eff}} = 3$ (symmetric, $\boldsymbol{a} = \boldsymbol{b} = \boldsymbol{l}$).

6.2 Algorithm

Algorithm 5 Off-diagonal metric

Require: learning rate η, momentum decay β_m, metric strength ξ, weight decay λ, inverse base metric γ^{-1}, mode choices for $\boldsymbol{a}$, $\boldsymbol{b}$, $\boldsymbol{l}$, $\boldsymbol{r}$

Initialise $t \leftarrow 0$, $\boldsymbol{m}_0 \leftarrow \mathbf{0}$
while θ_t not converged **do**
  $t \leftarrow t + 1$
  $\boldsymbol{g}_t \leftarrow \nabla_\theta \mathcal{L}(\theta_{t-1})$
  $\boldsymbol{m}_t \leftarrow \beta_m\, \boldsymbol{m}_{t-1} + (1 - \beta_m)\, \boldsymbol{g}_t$; $\hat{\boldsymbol{m}}_t \leftarrow \boldsymbol{m}_t/(1 - \beta_m^t)$
  Select $\boldsymbol{a}$, $\boldsymbol{b}$, $\boldsymbol{l}$, $\boldsymbol{r}$ from modes (gradient, momentum, θ, or $\mathbf{0}$)
  $U \leftarrow (\boldsymbol{a}, \boldsymbol{l})$; $V \leftarrow (\boldsymbol{l}, \boldsymbol{b} + \boldsymbol{l})$
  Compute c_{al}, c_{ll}, c_{ba}, c_{bl} and D via (17)
  Compute $G^{-1}\boldsymbol{r}$ via Woodbury (2×2 solve + $O(N)$ vector ops)
  $\theta_t \leftarrow \theta_{t-1} - \eta\, (G^{-1}\boldsymbol{r}) + \lambda\, \theta_{t-1}$
end while
return θ_t

Note that Algorithm 5 does not use a denominator EMA (β): the full 2×2 Woodbury solve is applied directly at each step.

6.3 Implementation

- JAX/Optax: `optimisers/jax_offdiag.py`
- PyTorch: `optimisers/torch_offdiag.py`

- Equivalence check: `optimisers/compare_offdiag.py`

Sweep names follow the pattern `sgd_offdiag_{a}_{b}` where {a} and {b} are drawn from `0` (zero), `l` (gradient), `m` (momentum), `theta` (parameters). All pairs except `sgd_offdiag_0_0` (which reduces to the fixed diagonal variant) are registered in `parameters/optimizer_registry.py`.

Computational cost. Taking $\gamma_{ij} = \gamma\,\delta_{ij}$ (scalar base metric) and writing

$$G = \gamma\,\mathbb{1}_N + \xi\,UV^\top, \qquad U = (\boldsymbol{a},\,\boldsymbol{l}), \qquad V = (\boldsymbol{l},\,\boldsymbol{b}+\boldsymbol{l}),$$

the Sherman-Morrison-Woodbury formula gives

$$G^{-1}\boldsymbol{r} = \frac{1}{\gamma}\boldsymbol{r} - \frac{\xi}{\gamma^2}\,U\left(\mathbb{1}_2 + \frac{\xi}{\gamma}V^\top U\right)^{-1} V^\top \boldsymbol{r}.$$

This costs $O(N)$: only inner products and a single 2×2 matrix inversion are required, never a full $N \times N$ matrix.

Modes for $\boldsymbol{l}$, $\boldsymbol{a}$, and $\boldsymbol{b}$. The vector $\boldsymbol{l}$ encodes the direction along which the loss coordinate stretches, with two built-in choices:

- **Gradient mode**: $\boldsymbol{l}_i = \partial_i\mathcal{L}$ — the metric is shaped by the instantaneous slope.
- **Momentum mode**: $\boldsymbol{l}_i = \boldsymbol{m}_i$ — the metric incorporates accumulated gradient history.

The off-diagonal coupling vectors $\boldsymbol{a}$ and $\boldsymbol{b}$ each support:

- **Zero**: $\boldsymbol{a}_i = 0$ or $\boldsymbol{b}_i = 0$ — eliminates one branch of the coupling.
- **Gradient**: aligned with $\partial_i\mathcal{L}$.
- **Momentum**: aligned with $\boldsymbol{m}_i$.
- **Parameter**: $\boldsymbol{a}_i = \theta^i$ or $\boldsymbol{b}_i = \theta^i$ — couples geometry to position.
- **Same-as-a**: $\boldsymbol{b}_i = \boldsymbol{a}_i$ — symmetric off-diagonal structure.

For full generality the implementations also accept user-provided vectors or callables.

7 Shared Implementation Details

7.1 Update Direction vs. Metric Direction

In all implementations, the goal is to realise

$$\delta\theta^i = -\eta\,g^{ij}r_j,$$

where r_j is the *update direction* and l_i is the direction used to *construct the metric*. These need not be the same vector. (Note that r_j here is a vector, not to be confused with the scalar damping factor r_t in Sections 3–5; the distinction is always clear from context and index type.) In ordinary gradient descent one would take $r_j = l_j = \partial_j\mathcal{L}$, but we also test momentum-based choices. The four combinations are:

- $\boldsymbol{l} = \nabla\mathcal{L},\ \boldsymbol{r} = \nabla\mathcal{L}$: pure geometric gradient descent.
- $\boldsymbol{l} = \nabla\mathcal{L},\ \boldsymbol{r} = \boldsymbol{m}$: metric shaped by the instantaneous gradient, step along momentum.
- $\boldsymbol{l} = \boldsymbol{m},\ \boldsymbol{r} = \nabla\mathcal{L}$: metric shaped by accumulated history, step along raw gradient.
- $\boldsymbol{l} = \boldsymbol{m},\ \boldsymbol{r} = \boldsymbol{m}$: both metric and step are momentum-based.

7.2 Computational Cost

All variants are $O(N)$ per step (assuming scalar or diagonal γ) and never materialise a full $N \times N$ matrix:

Variant	Dominant operation	Additional metric state
Fixed diagonal	scalar × vector	none
Learnable scalar	scalar × vector	1 float (s)
Learnable diagonal	element-wise vector × vector	N floats ($\boldsymbol{s}$)
Off-diagonal	inner products + 2×2 inverse	none

The metric-learning overhead for the learnable variants is negligible compared to the forward and backward passes.

7.3 Shared Hyperparameters

The following hyperparameters are common to the fixed diagonal, learnable scalar, and learnable diagonal variants:

- η (`lr`): base learning rate.
- β_m (`momentum`): EMA decay for the momentum buffer. (The paper [1] uses μ for this quantity; we write β_m to avoid collision with the metric learning rate μ.)
- ξ (`xi`): strength of the induced metric correction.
- β (`beta`): EMA decay for the denominator estimate $\hat{v}$.
- λ (`weight_decay`): decoupled weight decay ($\lambda \leq 0$ for decay, following the convention of [1]).

The off-diagonal variant uses η, β_m, ξ, and λ but does not use the denominator EMA β. The learnable variants additionally require:

- μ (`metric_lr`): step size for the online metric update.
- λ_s (`metric_reg`): L2 regularisation on s or $\boldsymbol{s}$.
- $s_{\max}$ (`metric_clip`): log-domain clipping bound on s, constraining $\alpha = e^s \in [e^{-s_{\max}}, e^{s_{\max}}]$.

8 Results

All optimisers are evaluated together on the same suite of benchmark tasks.

Results are compared head-to-head in the following notebooks:

- analysis/small_examples_analysis.ipynb — 2D synthetic functions (Beale, Rosenbrock, Himmelblau, Ackley, Rastrigin).
- `analysis/mnist_analysis.ipynb` — MNIST MLP (sweeps in progress).
- `analysis/cifar_analysis.ipynb` — CIFAR-10 ResNet-18 (sweeps in progress).
- `analysis/regression_analysis.ipynb` — Synthetic regression (sweeps in progress).
- `analysis/shake_analysis.ipynb` — Shakespeare language modelling (sweeps in progress).

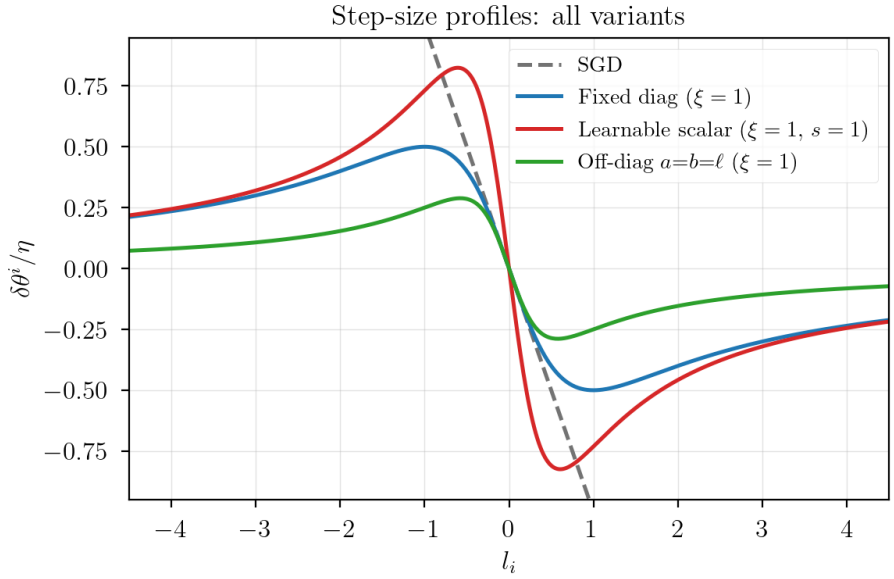

Figure 6: Step-size profiles compared across all variants in the single-parameter case ($N = 1$, $\gamma = 1$, $\xi = 1$). All induced-metric optimisers exhibit smooth gradient clipping relative to SGD. The learnable scalar ($s = 1$) has a larger peak step size; the off-diagonal symmetric coupling ($\boldsymbol{a} = \boldsymbol{b} = \boldsymbol{l}$) clips more aggressively.

Appendix A Repository Structure

A.1 Directory Layout

Directory	Contents
`optimisers/`	JAX and PyTorch optimiser implementations and equivalence checks
`parameters/`	Hyperparameter sweep scripts and shared training utilities
`analysis/`	Jupyter notebooks for loading and visualising sweep results
`documents/`	This document and supplementary notebooks
`images/`	Figures used in [1] and this document
`results/`	Local sweep results (JSON, gitignored)
`data/`	Local data files (gitignored)

A.2 Running Sweeps

Hyperparameter sweeps are driven by scripts in `parameters/` and support two backends: **local** (Optuna + JSON files, no account required) and **WandB** (cloud logging).

Script	Task	Model
`sweep_mnist_mlp.py`	MNIST classification	MLP (64 hidden)
`sweep_cifar10_resnet18.py`	CIFAR-10 classification	ResNet-18
`sweep_regression.py`	Synthetic regression	MLP
`sweep_shake.py`	Shakespeare language model	MiniGPT
`sweep_small_examples.py`	2D test functions	n/a

All scripts share a common CLI:

- `--optimiser`: optimiser name (see below).
- `--num_runs`: number of Optuna trials (default: 50).
- `--backend`: `local` or `wandb` (default: `wandb`).
- `--index`: run index for organising multiple runs of the same sweep.

Results are written to `results/<task>/<optimiser>/run_<index>/<run_id>.json`, each containing the full training history, final summary metrics, and the hyperparameter config used.

A.3 Available Optimisers

Sweep name	**Description**
`adam`, `adamw`, `sgd`, `muon`	Baselines
`sgd_metric`, `sgd_log_metric`, `sgd_rms`	Fixed diagonal metric
`sgd_learn_scalar`, `sgd_learn_scalar_log`	Learnable scalar metric
`sgd_learn_diag`, `sgd_learn_diag_log`	Learnable diagonal metric
`sgd_offdiag_{a}_{b}`	Off-diagonal metric ($\{0, l, m, \theta\}^2$ pairs, excluding 0_0)

The full optimiser registry is `parameters/optimizer_registry.py`.

A.4 Analysis Notebooks

Notebook	**Task**
`analysis/small_examples.ipynb`	2D test functions (Beale, Rosenbrock, etc.)
`analysis/mnist_analysis.ipynb`	MNIST MLP
`analysis/cifar_analysis.ipynb`	CIFAR-10 ResNet-18
`analysis/regression_analysis.ipynb`	Synthetic regression
`analysis/shake_analysis.ipynb`	Shakespeare language modelling

Each notebook supports both backends. Configure `BACKEND`, `TASK_TAG`, and `OPTIMIZERS` in the second cell. Outputs include training curves, 2D histograms of best metric vs. epoch, time-to-best analysis, and best-hyperparameter tables (also saved as CSV to `analysis/`).

References

[1] T. R. Harvey, "The Optimiser Hidden in Plain Sight: Training with the Loss Landscape's Induced Metric," *arXiv preprint arXiv:2509.03594*, 2025.

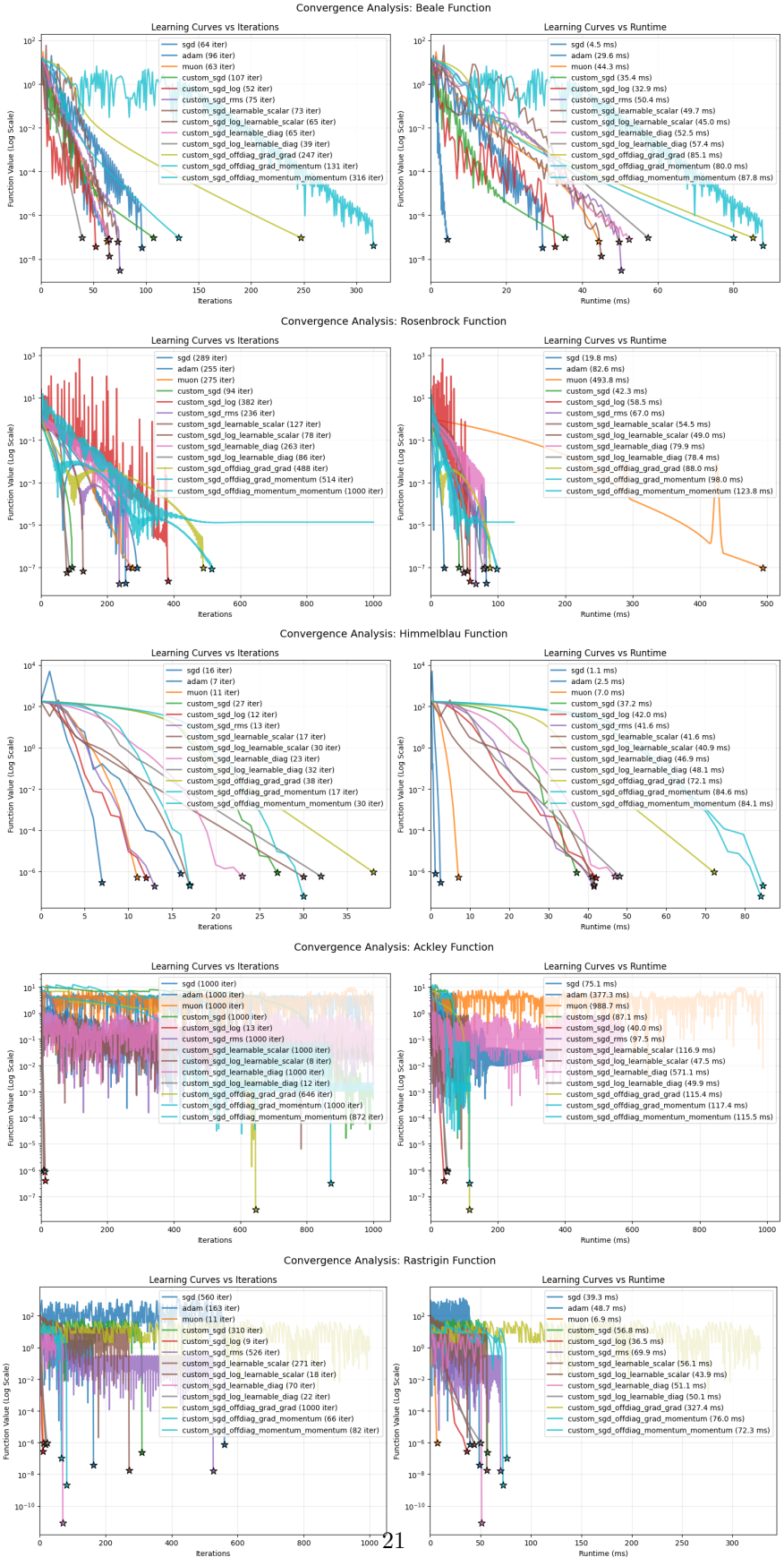

Figure 7: Small-example benchmark results for all optimiser variants. Each plot shows the best hyperparameter configuration found by the sweep. *(Fixed and off-diagonal sweeps are complete for small examples.)*